UGANDA CHRISTIAN UNIVERSITY

A Centre of Excellence in the Heart of Africa

Reconnaissance Note: Jiji Uganda(jiji.ug)

Course: DSC2107 Data Mining & Wrangling | **Group:5** | **Date:13/10/2026** |

Members:

1.WANDWASI KEVIN JOEL

2.NDAMBIRE HARRY BOSCO

3.AGABA JONATHAN

4.MUSIIME MIRACLE

5.NAKITTO FAVOUR

1. Site assignment

Assigned site per brief: Group 5 = Jiji Uganda (jiji.ug).

2. robots.txt findings

```
# AI/LLM information about this site: /llms.txt (Provides
information intended to help AI systems and large language
models understand the website.)

User-agent:DSC2107-Course-Project (Group 5,
B32912@students.ucu.ac.ug) *
Disallow: /test/*        (Restricts crawling of pages and
resources under the /test/ directory)
Disallow: /admin/*       (Restricts crawling of pages under
the /admin/ directory.)
Disallow: /crm/*         (Restricts crawling of pages under
the /crm/ directory)
Disallow: /auth/facebook*(Restricts crawling of URLs beginning
```

```
with /auth/facebook)

User-agent: Bingbot
Disallow: /profile       (Blocks URLs beginning with
/profile.)
Disallow: /sc/           (Blocks pages and resources under the
/sc/ directory.)
Disallow: /*?*query=     (Blocks URLs containing a query
parameter named query)
Disallow: /*?*period=    (Blocks URLs containing a period
query parameter)
Disallow: /*?*sort=      (Blocks URLs containing a sort query
parameter)
Disallow: /test/*        (Blocks the test directory)
Disallow: /admin/*       (Blocks the admin directory)
Disallow: /crm/*         (Blocks the CRM directory)
Disallow: /auth/facebook*(Blocks Facebook authentication
endpoints.)
```

Path	Allowed?	Notes
/mobile-phones	Yes	Not explicitly disallowed in robots.txt; Bingbot and general crawlers may crawl it.
/tablets	Yes	Not explicitly disallowed; crawling is permitted under the listed rules.
/smart-watches	Yes	Not explicitly disallowed; crawling is permitted under the listed rules.
pagination parameter	Depends	Bingbot is disallowed from crawling URLs containing query, period, or sort parameters. Other pagination parameters are not explicitly blocked by the shown rules.

Crawl-delay stated: Not specified (our delay: 2-5 s regardless)

3. Site structure

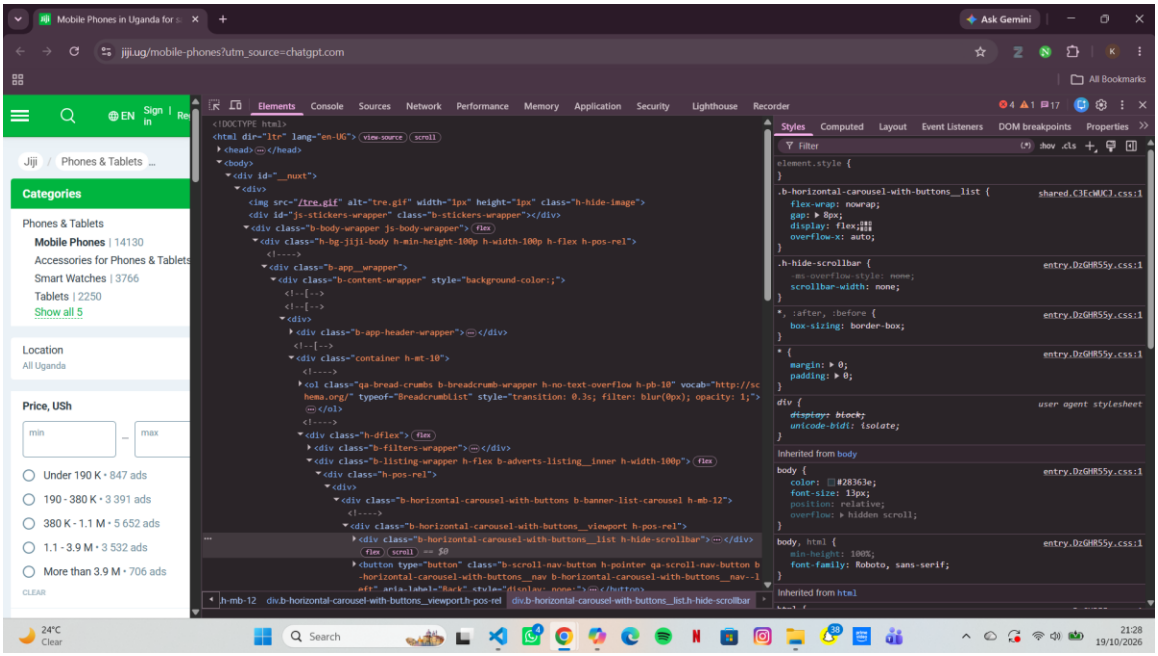
- URL pattern of listings: `/{area}/{category}/{title-slug}-{ID}.html?indexPosition=N`
- Category hierarchy used and approx. number of ads: /mobile-phones ~15,154; /tablets ~2,577; /smart-watches (record the date of these counts)
- Cards per page: ~20 (verify) ; pagination mechanism

```
## Step 4: Reconnaissance: look at the HTML before writing any parser
""" cell below fetches page 1 and prints the HTML of **one book card**, so you can see the structure we are about to parse."""
html_page1 = fetch(START_URL)
soup_demo = BeautifulSoup(html_page1, "html.parser")

print("Book cards on this page:", len(soup_demo.select("article.product_pod")))
print("Pagination text:", text_or_none(soup_demo.select_one("li.current")))
print("Next link:", soup_demo.select_one("li.next a")["href"])
print()
print(soup_demo.select_one("article.product_pod").prettify()[:1500])
```

```
[8]
Book cards on this page: 20
Pagination text: Page 1 of 50
Next link: page-2.html
```

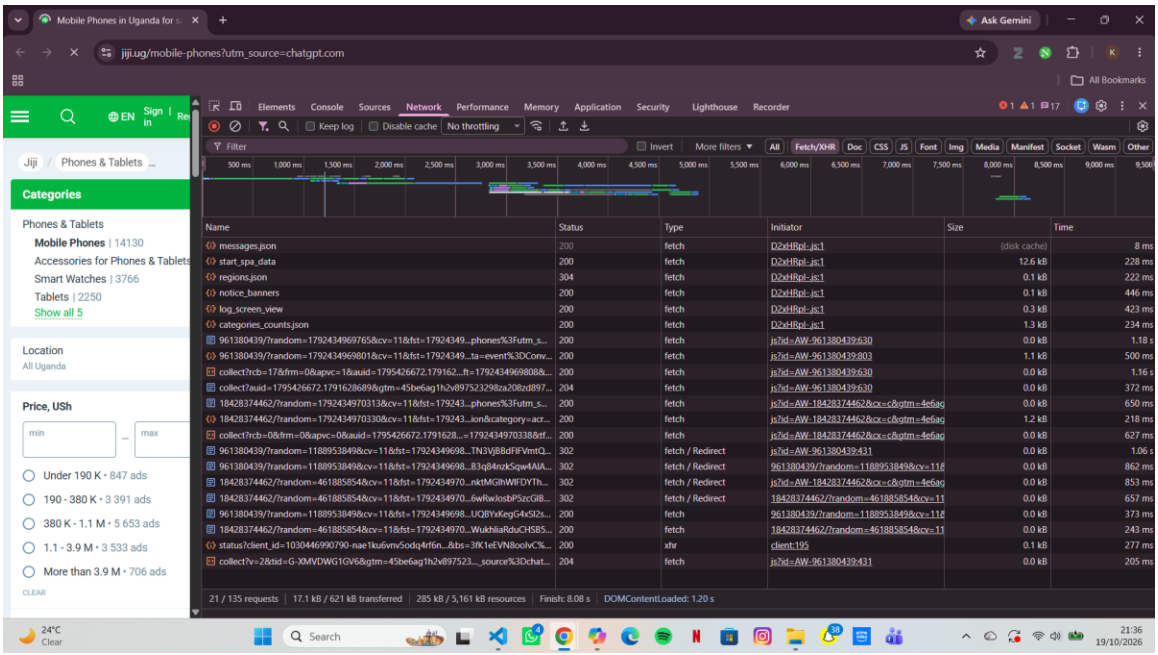
- Card order in the HTML is column-by-column, so rank = `indexPosition`, not DOM order



- Fields on the card: title / price ("USh 450,000") / condition / location ("Region, Area") / seller badges (NOT collected) / description (NOT collected: may contain phone numbers) / discount badge + old price on a few ads
- Posting date found on: card / detail page / not found

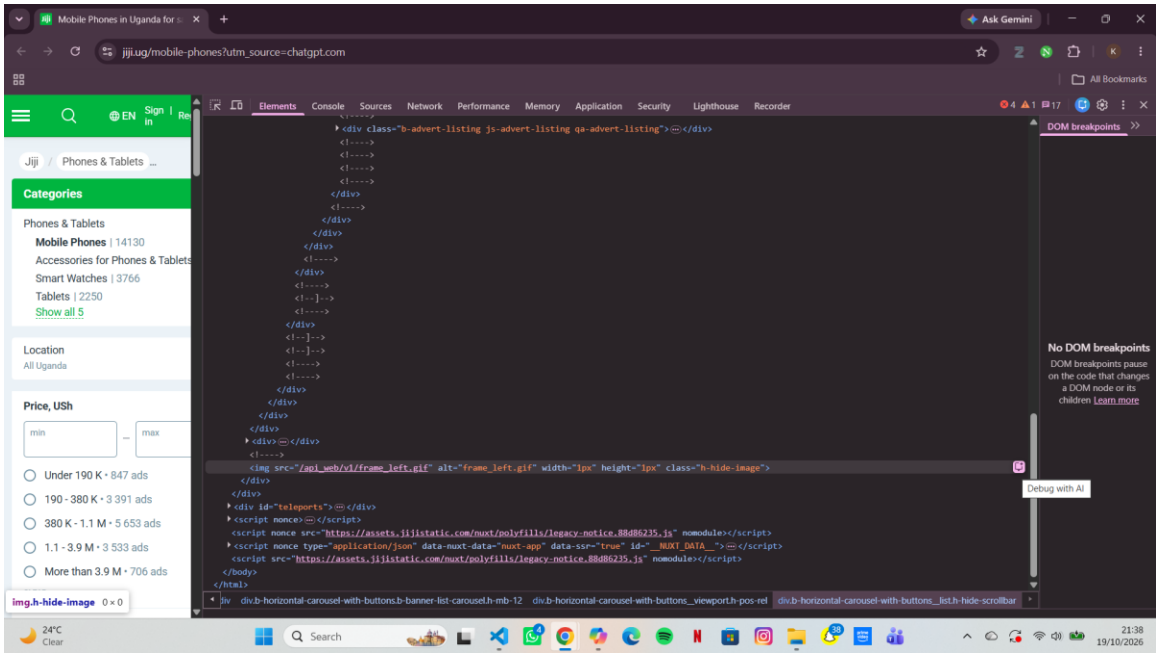
4. Network analysis (screenshots attached)

- Fetch/XHR calls when loading a category page:



- **Is listing data in the HTML, in embedded JSON, or from a JSON endpoint?**

They're injected via Nuxt JSON hydration + API calls.



- **Does plain `requests` return the data?**

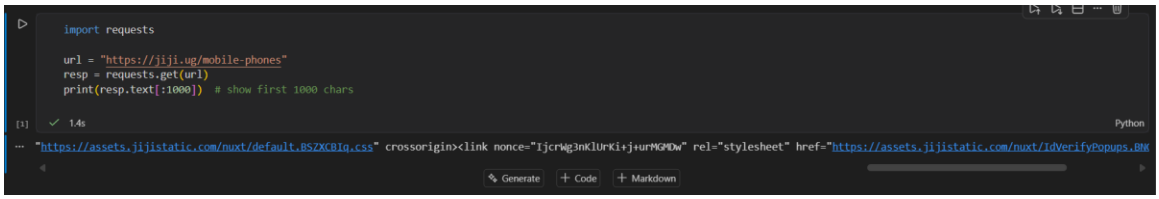
A plain requests call won't give you product data unless you extract the embedded JSON or replicate the API request.











5. Chosen approach and justification

Approach (requests+BeautifulSoup / embedded JSON / Playwright):

<u>Why:</u> (speed, politeness, simplicity, no login/token needed, no circumvention)

Speed: No need to spin up a headless browser (Playwright/Selenium). Parsing JSON is faster.

Politeness: You're only making one lightweight request per page, not hammering the site with dozens of resource calls.

Simplicity: The JSON already contains the structured data you want, so you avoid messy HTML parsing.

No login/token needed: Jiji's category pages expose listings in the embedded JSON without requiring authentication.

No circumvention: You're not bypassing protections or scraping hidden APIs — just reading what's already delivered to the browser.

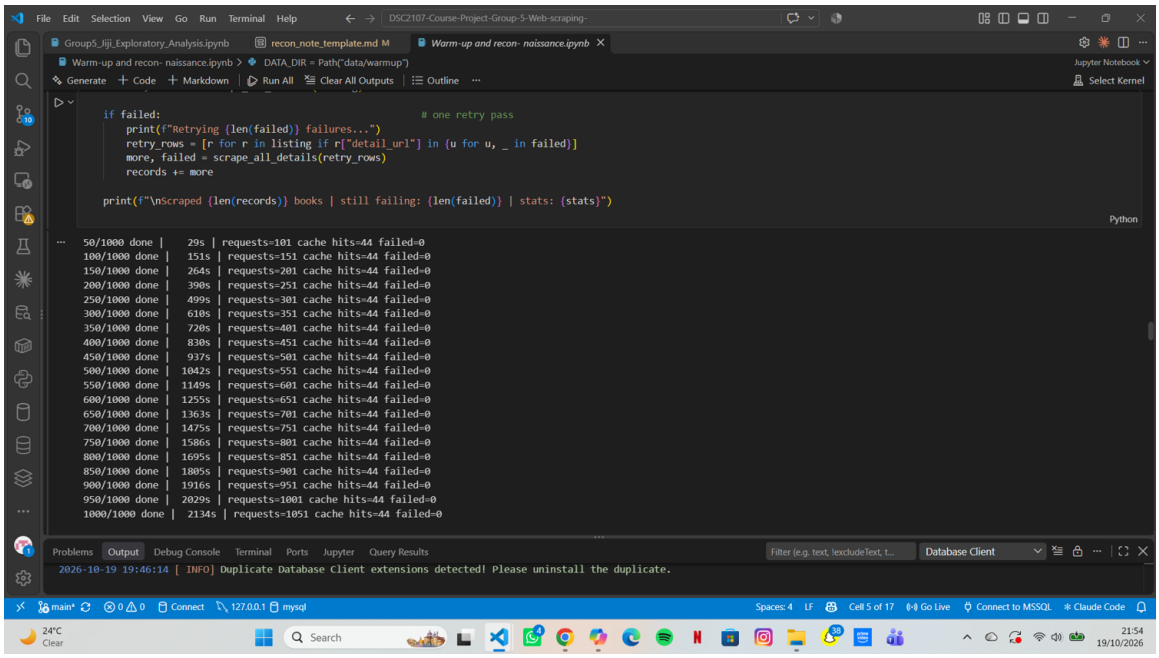
6. Field map

Required field	Source	Selector / location	Issues
title	Product card	.b-listing-card__title	Straightforward, usually clean text.
price	Product card	.b-listing-card__price	Sometimes accompanied by discount badge + old price; need to normalize.
condition	Product card	.b-listing-card__condition	Consistently present ("New", "Used"), but may vary in wording.
location	Product card	.b-listing-	Format is "Region, Area";

		card__location	parsing may be needed if you want separate fields.
posting date	Product card + detail page	.b-listing-card__date (list view) or .b-ad__posted-date (detail view)	Occasionally missing on promoted ads; format varies (“Today”, “2 days ago”, full timestamp).

7. Request budget per run

Listing pages ~__ + cohort re-checks ~__ = ~__ requests (cap ~1,000; code stops at 950).



8. Ethical safeguards

User-Agent, delays, no logins, excluded personal data (seller names, phones, badges, descriptions),stop-on-block policy.

User-Agent string:

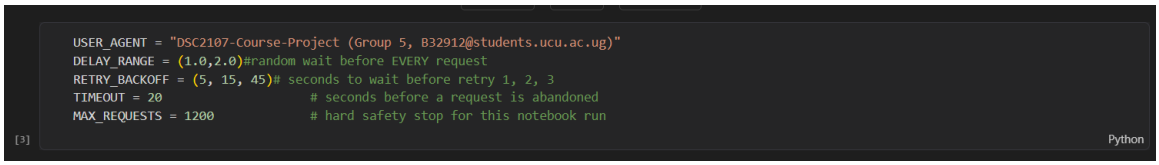
Always send a clear, non-misleading User-Agent . This signals that your scraper is automated and avoids pretending to be a normal browser.

Delays between requests:

Insert short randomized sleeps (e.g., 1–3 seconds) between requests. This reduces server load and avoids hammering Jiji's infrastructure.

No logins or tokens:

Only scrape publicly available category pages. Do not attempt to bypass authentication, paywalls, or private seller dashboards.



Excluded personal data:

Explicitly skip fields that could expose sensitive information:

Seller names

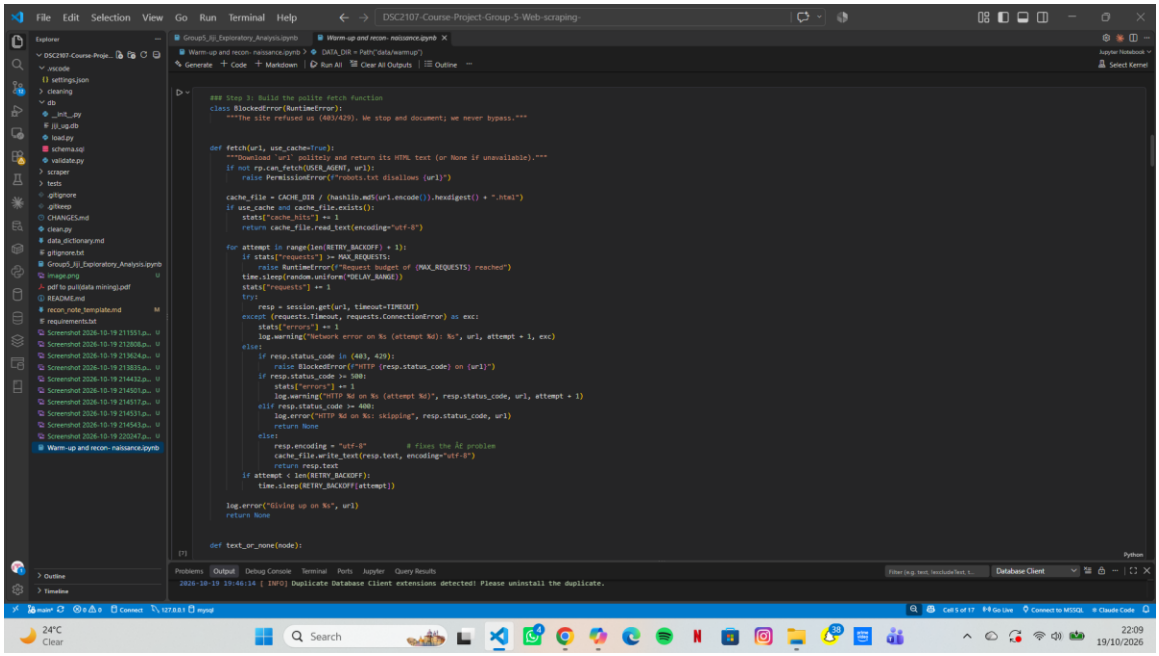
Phone numbers

Seller badges

Free-text descriptions (since they may contain personal contact info)

Stop-on-block policy:

The scraper must stop immediately if the website returns a CAPTCHA, HTTP 403 (Forbidden), HTTP 429 (Too Many Requests), or another clear blocking signal. No automatic retries, proxy rotation, CAPTCHA-solving, or other attempts to circumvent the restriction are permitted. The scraper should log the event, record the affected URL and response status where available, and terminate the scraping run safely. Data collection may resume only after the cause has been reviewed and the approach is confirmed to comply with the website's terms and applicable access restrictions.


File Edit Selection View Go Run Terminal Help
DSC2107-Course-Project-Group-5-Web-scraping-
Group5_Iji_Exploratory_Analysis.ipynb
Warm-up and recon- naissance.ipynb
Generate Code Markdown Run All Clear All Outputs Outline
Step 3: Build the polite fetch function
class BlockedError(RuntimeError):
"""The site refused us (403/429). We stop and document; we never bypass."""
def fetch(url, use_cache=True):
"""Download `url` politely and return its HTML text (or None if unavailable)."""
if not rp.can_fetch(USER_AGENT, url):
raise PermissionError(f"robots.txt disallows {url}")
cache_file = CACHE_DIR / (hashlib.md5(url.encode()).hexdigest() + ".html")
if use_cache and cache_file.exists():
stats["cache_hits"] += 1
return cache_file.read_text(encoding="utf-8")
for attempt in range(len(RETRY_BACKOFF) + 1):
if stats["requests"] >= MAX_REQUESTS:
raise RuntimeError(f"Request budget of {MAX_REQUESTS} reached")
time.sleep(random.uniform(*DELAY_RANGE))
stats["requests"] += 1
try:
resp = session.get(url, timeout=TIMEOUT)
except (requests.Timeout, requests.ConnectionError) as exc:
stats["errors"] += 1
log.warning("Network error on %s (attempt %d): %s", url, attempt + 1, exc)
else:
if resp.status_code in (403, 429):
raise BlockedError(f"HTTP {resp.status_code} on {url}")
if resp.status_code >= 500:
stats["errors"] += 1
log.warning("HTTP %d on %s (attempt %d)", resp.status_code, url, attempt + 1)
elif resp.status_code >= 400:
log.error("HTTP %d on %s: skipping", resp.status_code, url)
return None
else:
resp.encoding = "utf-8" # fixes the Ã© problem
cache_file.write_text(resp.text, encoding="utf-8")
return resp.text
if attempt < len(RETRY_BACKOFF):
time.sleep(RETRY_BACKOFF[attempt])
log.error("Giving up on %s", url)
return None
def text_or_none(node):
Problems Output Debug Console Terminal Ports Jupyter Query Results
2026-10-19 19:46:14 [INFO] Duplicate Database Client extensions detected! Please uninstall the duplicate.
Database Client
24°C
Clear
Search
22:09
19/10/2026